SQL Advanced

Objective

Now that you are comfortable with the basics, this part covers the techniques that separate a confident SQL user from a beginner: advanced joins, subqueries, window functions, CTEs, views, performance, and transactions. We continue using the same Employees and Departments tables.

Outline

1. Advanced Joins
   - 1.1. FULL OUTER JOIN
   - 1.2. SELF JOIN
   - 1.3. CROSS JOIN
2. Subqueries
   - 2.1. Subquery in WHERE
   - 2.2. Subquery in FROM
   - 2.3. Correlated Subqueries
   - 2.4. Key Points
3. Set Operations
4. Window Functions
   - 4.1. Common Window Functions
   - 4.2. GROUP BY vs Window Functions
5. Common Table Expressions (CTEs)
   - 5.1. Basic CTE
   - 5.2. Recursive CTE
   - 5.3. CTE vs Subquery
6. Views
   - 6.1. Creating a View
   - 6.2. Key Points
7. Indexes and Query Performance
   - 7.1. What is an Index?
   - 7.2. Common Types of Indexes
   - 7.3. When Should You Avoid Creating an Index?
   - 7.4. Reading a Query Plan (EXPLAIN)
8. Stored Procedures and Functions
   - 8.1. Stored Procedures
   - 8.2. Functions
   - 8.3. Stored Procedure vs Function
9. Transactions
   - 9.1. Transaction Control Commands
   - 9.2. ACID Properties
10. SQL Triggers
    - 10.1. Purpose of Triggers
    - 10.2. Types of Triggers
11. Performance Tuning
    - 11.1. Why is it Important?

11.2. Best Practices
12. Summary
13. Reference Material
14. Advanced Reading Material

1. Advanced Joins: FULL OUTER, SELF, and CROSS

FULL OUTER JOIN

Returns everything: matching rows, unmatched rows from the left table, and unmatched rows from the right table. Missing values on either side show as NULL.

```
SELECT e.emp_name, d.department_name
FROM Employees e
FULL OUTER JOIN Departments d
   ON e.department_id = d.department_id;
```

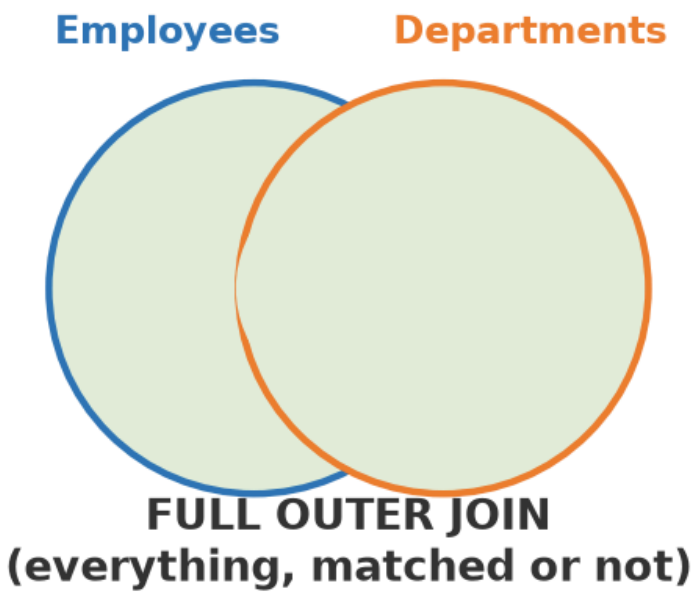


Figure 6: FULL OUTER JOIN keeps every row from both tables, matched or not.

Note: *MySQL does not support FULL OUTER JOIN directly - it can be simulated with a LEFT JOIN UNION a RIGHT JOIN.*

SELF JOIN

A self join joins a table to itself - useful when rows relate to other rows in the same table, such as an employee and their manager (who is also an employee).

```
SELECT e.emp_name AS employee, m.emp_name AS manager
```

```
FROM Employees e
LEFT JOIN Employees m
   ON e.manager_id = m.emp_id;
```

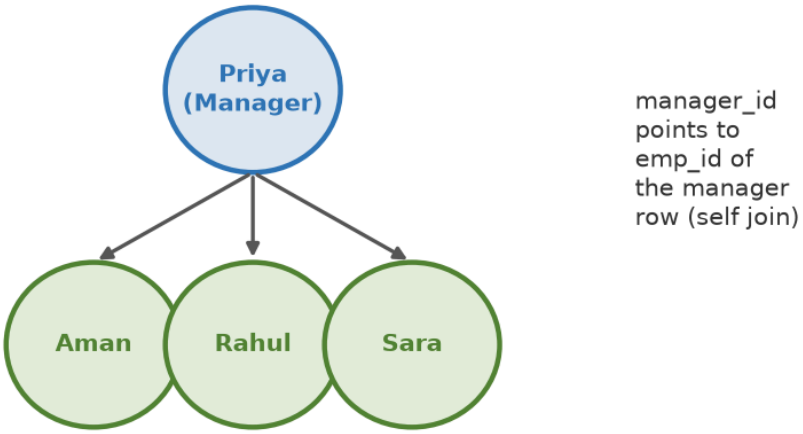


Figure 7: A self join matches each employee's manager_id to another row's emp_id in the very same table.

employee	manager
Priya Sharma	NULL
Amit Verma	Priya Sharma
Neha Gupta	Priya Sharma
Rahul Singh	Priya Sharma
Sara Khan	Rahul Singh

CROSS JOIN

A cross join returns every possible combination of rows from two tables (the Cartesian product), with no matching condition. It is used sparingly - for example, to generate all combinations of products and sizes.

```
SELECT e.emp_name, d.department_name
FROM Employees e
CROSS JOIN Departments d;   -- 7 employees x 4 departments = 28 rows
```

2. Subqueries

A subquery is a query nested inside another query. It runs first, and its result is used by the outer query.

Subquery in WHERE

```
SELECT emp_name, salary
FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees);
```

Note: *This returns every employee earning more than the company-wide average salary - the subquery calculates that average first.*

Subquery in FROM

```
SELECT department_id, avg_salary
FROM (
   SELECT department_id, AVG(salary) AS avg_salary
   FROM Employees
   GROUP BY department_id
) AS dept_avg
WHERE avg_salary > 65000;
```

Correlated Subqueries

A correlated subquery references a column from the outer query, so it re-runs once per outer row - powerful, but can be slower on large tables.

```
SELECT emp_name, salary, department_id
FROM Employees e1
WHERE salary = (
   SELECT MAX(salary)
   FROM Employees e2
   WHERE e2.department_id = e1.department_id
);   -- top earner in each department
```

Key Points

- The inner query executes before the outer query.
- A subquery is always enclosed in parentheses ().
- It can return a single value, multiple values, or a result set.
- Subqueries make SQL queries simpler and more readable.

3. Set Operations: UNION, INTERSECT, EXCEPT

Set operations combine the results of two separate SELECT statements that return the same number and type of columns.

Operator	Returns
UNION	All distinct rows from both queries (duplicates removed)
UNION ALL	All rows from both queries, including duplicates (faster)
INTERSECT	Only rows that appear in both queries
EXCEPT / MINUS	Rows in the first query that do not appear in the second

```
SELECT emp_name FROM Employees WHERE department_id = 1
UNION
SELECT emp_name FROM Employees WHERE salary > 90000;
```

Since this guide uses MySQL for hands-on practice, it is important to know that INTERSECT and EXCEPT are not supported in MySQL. However, similar results can be achieved using INNER JOIN and LEFT JOIN.

1. **INTERSECT Alternative** (Finding Common Rows)
INTERSECT returns rows that are present in both result sets. In MySQL, you can achieve the same result using an INNER JOIN.

```
SELECT DISTINCT e1.emp_name
FROM Employees e1
INNER JOIN Employees e2
   ON e1.emp_id = e2.emp_id
WHERE e1.department_id = 1
 AND e2.salary > 90000;
```

2. **EXCEPT Alternative** (Finding Unique Rows)
EXCEPT returns rows from the first result set that are not present in the second result set. In MySQL, you can achieve the same result using a LEFT JOIN with IS NULL.

```
SELECT DISTINCT e1.emp_name
FROM Employees e1
LEFT JOIN Employees e2
   ON e1.emp_id = e2.emp_id
   AND e2.salary > 90000
WHERE e1.department_id = 1
 AND e2.emp_id IS NULL;
```

4. Window Functions

Window functions perform a calculation across a set of rows related to the current row - without collapsing them into a single result like GROUP BY does. This makes them ideal for rankings, running totals, and row-by-row comparisons.

```
SELECT
   emp_name,
   department_id,
   salary,
   RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS salary_rank
FROM Employees;
```

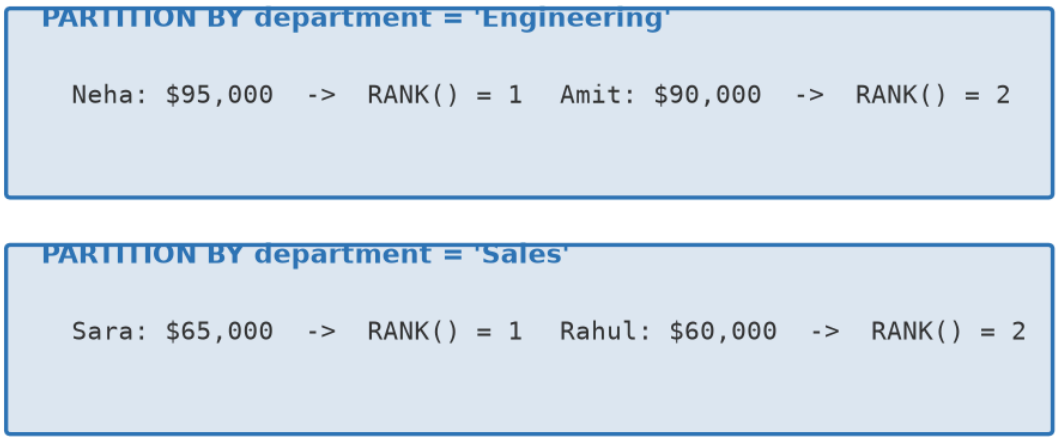


Figure 8: PARTITION BY splits rows into groups (here, by department); ORDER BY then ranks the rows inside each group.

emp_name	department_id	salary	salary_rank
Neha Gupta	1	95000	1
Amit Verma	1	90000	2
Sara Khan	2	65000	1
Rahul Singh	2	60000	2
Vikram Rao	3	70000	1
Anjali Nair	4	72000	1

Common Window Functions

Function	Purpose
ROW_NUMBER()	Assigns a unique sequential number to each row in the partition
RANK()	Assigns a rank; ties share a rank and a gap follows
DENSE_RANK()	Assigns a rank; ties share a rank with no gap after
SUM()/AVG() OVER()	Running totals or moving averages
LAG()	Returns the value from the previous row in the same partition.

Example: Using Multiple Window Functions

The following query demonstrates how multiple window functions can be used together in a single query.

```
SELECT
  emp_name,
```

```
  department_id,
  salary,

  RANK() OVER (
     PARTITION BY department_id
     ORDER BY salary DESC
  ) AS salary_rank,

  LAG(salary) OVER (
     PARTITION BY department_id
     ORDER BY salary DESC
  ) AS previous_salary,

  SUM(salary) OVER (
     PARTITION BY department_id
     ORDER BY salary DESC
  ) AS running_total

FROM Employees;
```

Sample Output

emp_name	department_id	salary	salary_rank	previous_salary	running_total
Priya Sharma	1	150000	1	NULL	150000
Neha Gupta	1	95000	2	150000	245000
Amit Verma	1	90000	3	95000	335000
Sara Khan	2	65000	1	NULL	65000
Rahul Singh	2	60000	2	65000	125000

GROUP BY vs Window Functions

Both GROUP BY and Window Functions perform calculations on data, but they differ in how they return the results.

Feature	GROUP BY	Window Functions
Result	Returns one row per group.	Returns all original rows with additional calculated values.
Rows	Reduces the number of rows.	Retains all rows.

Use Case	Calculate summary values for each group.	Calculate rankings, running totals, or averages while displaying individual rows.

Example using GROUP BY

```
SELECT department_id,
    AVG(salary) AS avg_salary
FROM Employees
GROUP BY department_id;
```

Result: Returns one row for each department with its average salary.

Example using a Window Function

```
SELECT emp_name,
    department_id,
    salary,
    AVG(salary) OVER (PARTITION BY department_id) AS avg_salary
FROM Employees;
```

Result: Returns all employees, along with the average salary of their respective department.

5. Common Table Expressions (CTEs)

A CTE, written with **WITH**, defines a temporary, named result set that you can reference later in the same query - like giving a subquery a name to make long queries far easier to read.

```
WITH dept_avg AS (
  SELECT department_id, AVG(salary) AS avg_salary
  FROM Employees
  GROUP BY department_id
)
SELECT e.emp_name, e.salary, d.avg_salary
FROM Employees e
JOIN dept_avg d ON e.department_id = d.department_id
WHERE e.salary > d.avg_salary;
```

Recursive CTEs

A recursive CTE repeatedly references itself to walk through hierarchical data - a perfect fit for our manager-employee reporting chain.

```
WITH RECURSIVE org_chart AS (
  SELECT emp_id, emp_name, manager_id, 1 AS level
  FROM Employees
  WHERE manager_id IS NULL        -- start at the top manager

  UNION ALL

  SELECT e.emp_id, e.emp_name, e.manager_id, oc.level + 1
  FROM Employees e
  JOIN org_chart oc ON e.manager_id = oc.emp_id
)
SELECT * FROM org_chart ORDER BY level;
```

Key Points

- A CTE is created using the **WITH** keyword.
- It exists only for the execution of the query.
- CTEs improve the readability and maintainability of complex SQL queries.
- Recursive CTEs are useful for working with hierarchical data such as employee-manager relationships.

When Should You Use a CTE Instead of a Subquery

Both CTEs (Common Table Expressions) and Subqueries help break complex SQL queries into smaller parts. However, a CTE is often preferred when writing long or complex queries because it improves readability and makes the query easier to maintain..

Feature	Subquery	CTE
Readability	Can become difficult to read when deeply nested.	Easier to read and understand because it has a meaningful name.
Reusability	Must be written again if needed multiple times.	Can be referenced multiple times within the same query.
Complex Queries	Suitable for simple queries.	Better suited for complex queries involving multiple steps.
Recursive Queries	Not supported.	Supports recursive queries using WITH RECURSIVE

6. Views

A **View** is a virtual table created from the result of a SQL query. It does not store data itself; instead, it stores the query definition. Whenever you query a view, SQL executes the underlying SELECT statement and returns the latest data.

Views are commonly used to simplify complex queries, improve code reusability, and provide restricted access to specific data.

```
CREATE VIEW high_earners AS
SELECT emp_name, department_id, salary
FROM Employees
WHERE salary > 80000;

-- Now query it just like a table:
SELECT * FROM high_earners;
```

Key Points

- A view stores only the SQL query, not the actual data.
- It always displays the latest data from the underlying tables.
- Views simplify complex queries and promote query reuse.
- They can be used to hide sensitive columns and provide controlled access to data.
- Views help maintain consistency by allowing everyone to use the same query definition.

7. Indexes and Query Performance

An **Index** is a database object that improves the speed of data retrieval operations. It works like the index in a book, allowing the database to quickly locate rows without scanning the entire table.

Common Types of Indexes

Primary Key Index – Created automatically for a primary key.
Unique Index – Ensures all values in a column are unique.
Composite Index – Created on multiple columns.
Clustered/Non-Clustered Index – Organize and store data differently (database-specific).

```
CREATE INDEX idx_employees_department
ON Employees (department_id);
```

Key Points

- Indexes speed up data retrieval by reducing table scans.
- They are most effective on columns that are frequently used in WHERE, JOIN, ORDER BY, and GROUP BY clauses.
- Too many indexes can slow down INSERT, UPDATE, and DELETE operations.

When Should You Avoid Creating an Index

Although indexes improve query performance, creating an index is not always the best choice. In some situations, an index can consume extra storage and slow down data modification operations.
Avoid creating indexes in the following situations:

- Small Tables - For tables containing only a small number of rows, a full table scan is often faster than using an index.
- Columns with Low Cardinality - Columns that have very few distinct values (such as Gender or Status) usually do not benefit from indexing.
- Frequently Updated Columns - Columns that are updated very often can experience slower INSERT, UPDATE, and DELETE operations because the index must also be updated.

Reading a Query Plan

Most databases let you preview how a query will run using EXPLAIN (or EXPLAIN PLAN). This shows whether the database used an index or scanned the whole table - a full table scan on a huge table is usually a sign to add an index.

```
EXPLAIN SELECT * FROM Employees WHERE department_id = 1;
```

8. Stored Procedures and Functions

Stored Procedures

A **Stored Procedure** is a precompiled collection of SQL statements stored in the database. It is used to perform one or more database operations and can accept input parameters. Stored procedures help improve code reusability, performance, and maintainability.

```
CREATE PROCEDURE GetEmployeesByDept (IN dept_id INT)
BEGIN
   SELECT emp_name, salary
   FROM Employees
   WHERE department_id = dept_id;
END;

- Call it:
CALL GetEmployeesByDept(1);
```

Key Points

- Stored procedures are created using the CREATE PROCEDURE statement.
- They can accept input and output parameters.
- They can contain multiple SQL statements.
- They are executed using EXEC or CALL (depending on the database).

Functions

A **Function** is a reusable database object that performs a specific task and always returns a value. Functions are commonly used to perform calculations, manipulate data, or return computed values. Unlike stored procedures, functions can be used directly inside SQL statements such as SELECT, WHERE, and ORDER BY.

```
CREATE FUNCTION GetAnnualSalary(monthly_salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
BEGIN
        RETURN monthly_salary * 12;
```

```
END;

- Call it:
SELECT emp_name,
        GetAnnualSalary(salary) AS annual_salary
FROM Employees;
```

Stored Procedure vs Function

Feature	Stored Procedure	Function
Purpose	Performs one or more database operations.	Performs a calculation or returns a value.
Return Value	May or may not return a value.	Always returns a value.
Invocation	Called using CALL or EXEC.	Used inside SQL statements like SELECT.
Typical Use Case	Insert, update, delete, or execute business logic.	Perform calculations, format data, or return computed values.

Note: *The syntax for stored procedures and functions may vary slightly across database systems such as MySQL, SQL Server, and Oracle. Always refer to the documentation of your specific database.*

9. Transactions

A Transaction is a group of one or more SQL statements that are executed as a single unit of work. It ensures that either all the operations are completed successfully or none of them are applied. This helps maintain data accuracy and consistency.

For example, when transferring money from one bank account to another, both the debit and credit operations must succeed. If one operation fails, the entire transaction is rolled back.

SQL Transaction Control Commands

Transaction Control Commands are used to manage transactions and control when changes are saved or undone.

1. BEGIN TRANSACTION : Starts a new transaction.

```
BEGIN TRANSACTION;

UPDATE Employees
SET salary = 55000
WHERE emp_id = 101;
```

The salary is updated temporarily until the transaction is committed.

2. COMMIT : Permanently saves all changes made during the transaction.

```
BEGIN TRANSACTION;

UPDATE Employees
SET salary = 55000
WHERE emp_id = 101;

COMMIT;
```

The salary update is permanently stored in the database.

3. ROLLBACK : Cancels all changes made during the current transaction.

```
BEGIN TRANSACTION;

UPDATE Employees
SET salary = 55000
WHERE emp_id = 101;

ROLLBACK;
```

The salary remains unchanged because the transaction is undone.

4. SAVEPOINT : Creates a checkpoint within a transaction, allowing partial rollback.

```
BEGIN TRANSACTION;

UPDATE Employees
SET salary = 55000
WHERE emp_id = 101;

SAVEPOINT sp1;

DELETE FROM Employees
WHERE emp_id = 102;
```

sp1 acts as a checkpoint inside the transaction.

ACID Properties

Every transaction follows the ACID properties to ensure data reliability:

Property	Meaning
Atomicity	All statements in the transaction succeed, or none do
Consistency	The database moves from one valid state to another

Isolation	Concurrent transactions do not interfere with each other
Durability	Once committed, changes survive even a system crash

Understanding ACID with a Real-World Example

Suppose a bank customer transfers ₹5,000 from Account A to Account B.

The transaction consists of two operations:

Before Transaction

Account	Balance
A	₹20,000
B	₹10,000

After Successful Transaction

Account	Balance
A	₹15,000
B	₹15,000

```
BEGIN TRANSACTION;

-- Step 1: Deduct ₹5,000 from A's account
UPDATE Accounts
SET balance = balance - 5000
WHERE account_id = 'A101';

-- Step 2: Add ₹5,000 to B's account
UPDATE Accounts
SET balance = balance + 5000
WHERE account_id = 'B101';

COMMIT;
```

How ACID Applies

1. Atomicity

Both operations are treated as one transaction.

If ₹5,000 is deducted from Account A but cannot be credited to Account B, the deduction is rolled back.

This prevents partial updates.

Result: Either both operations succeed or neither does.

2. Consistency

The database always remains in a valid state.

Before the transfer, the total balance is ₹30,000.

After the transfer, the total balance is still ₹30,000.

Result: No money is lost or created during the transaction.

3. Isolation

Suppose two customers transfer money at the same time.

Each transaction is processed independently, so one transaction cannot see incomplete changes made by another.

Result: Concurrent transactions do not interfere with each other.

4. Durability

Once the transaction is successfully committed, the updated balances are permanently saved.

Even if the database server crashes immediately after the commit, the balances remain:

Account	Balance
A	₹15,000
B	₹15,000

Result: Committed changes are never lost.

10. SQL Triggers

A Trigger is a special type of stored procedure that executes automatically when a specific event occurs in the database. Unlike stored procedures, triggers do not need to be called manually. They are commonly used for auditing, data validation, enforcing business rules, and maintaining data integrity.

Purpose of Triggers

- Automate repetitive tasks.
- Maintain audit logs.
- Enforce business rules.
- Improve data integrity.

Note: *The examples in this section use SQL Server syntax. Trigger syntax may vary across different database systems such as MySQL, PostgreSQL, Oracle, and SQL Server.*

Types of SQL Triggers

1. DML Trigger (Data Manipulation Language)

A DML Trigger executes when data in a table is INSERTED, UPDATED, or DELETED.

Example: Track New Orders

```
CREATE TRIGGER trgAfterInsertOrder
ON Orders
AFTER INSERT
AS
BEGIN
        INSERT INTO OrderLog(OrderID, ActionDate)
        SELECT OrderID, GETDATE()
        FROM inserted;
END;
```

Result: Whenever a new order is placed, the trigger automatically stores the order details in the OrderLog table.

Common Use Cases : Audit logging, Tracking salary changes, Maintaining history tables.

2. DDL Trigger (Data Definition Language)

A DDL Trigger executes whenever the database structure changes, such as CREATE, ALTER, or DROP operations.

Example: Prevent Table Deletion

```
CREATE TRIGGER trgPreventTableDrop
ON DATABASE
FOR DROP_TABLE
AS
BEGIN
        PRINT 'Dropping tables is not allowed.';
        ROLLBACK;
END;
```

Result: If someone tries to delete a table, the trigger cancels the operation.

Common Use Cases : Prevent accidental table deletion, Audit schema changes, Protect important database objects.

3. Logon Trigger

A Logon Trigger executes whenever a user logs in to the SQL Server instance.

Example: Restrict Login Time

```
CREATE TRIGGER trgLimitLogonTime
ON ALL SERVER
FOR LOGON
AS
BEGIN
        IF DATEPART(HOUR, GETDATE()) NOT BETWEEN 9 AND 17
        ROLLBACK;
END;
```

Result : Login at 10:00 AM → Allowed.

Login at 8:00 PM → Denied.

Common Use Cases : Restrict login during maintenance, Audit login activity, Improve database security.

11. Performance Tuning

SQL Performance Tuning is the process of optimizing SQL queries to improve their execution speed and use database resources efficiently.

Why is it Important?

- Improves query performance.
- Reduces CPU and memory usage.
- Enhances application performance.
- Handles large datasets efficiently.

Best Practices

1. Use Indexes

Create indexes on columns frequently used in WHERE, JOIN, and ORDER BY clauses**.**

```
CREATE INDEX idx_emp_department
ON Employees(department_id);
```

2. Avoid SELECT *

Retrieve only the required columns.

```
 SELECT emp_id, emp_name
FROM Employees;
```

3. Filter Data

Use the WHERE clause to retrieve only the required rows.

```
SELECT *
FROM Employees
WHERE department_id = 1;
```

4. Use EXPLAIN

Analyze how the database executes a query.

```
EXPLAIN
SELECT *
FROM Employee
Where department_id =1;
```

5. Avoid Unnecessary DISTINCT

Use DISTINCT only when you need to remove duplicate rows, as it adds extra processing.

```
SELECT DISTINCT department_id
FROM Employees;
```

6. Avoid Functions on Indexed Columns

Avoid applying functions to indexed columns because it can prevent the database from using the index efficiently.
Instead of:

```
SELECT *
FROM Employees
WHERE YEAR(hire_date) = 2020;
```

Use:

```
SELECT *
FROM Employees
WHERE hire_date BETWEEN '2020-01-01' AND '2020-12-31';
```

7. Retrieve Only Required Rows

Use the WHERE clause to retrieve only the rows you need.

```
SELECT *
FROM Employees
WHERE department_id = 1;
```

12. Summary

In this module, we explored advanced SQL concepts used to solve complex data retrieval and analysis problems. We covered advanced joins, subqueries, set operations, window functions, Common Table Expressions (CTEs), views, indexes, stored procedures, functions, transactions, triggers, and performance tuning techniques. These concepts help us write efficient, maintainable, and optimized SQL queries for real-world database applications.

13. Gem

- https://gemini.google.com/gem/1K7z_Hgar7nqLb-3lqSFRt6Ua557kE2jW?usp=sharing

14. Reference material

- W3Schools SQL Tutorial - https://www.w3schools.com/sql/
- PostgreSQL Official Documentation - https://www.postgresql.org/docs/
- Mode Analytics SQL Tutorial - https://mode.com/sql-tutorial/

15. Advanced reading material (good to have)

- Use The Index, Luke! (query performance & indexing) - https://use-the-index-luke.com/
- Modern SQL - https://modern-sql.com/
- SQL Window Functions Explained - https://www.sqlwindowfunctions.com/